

~/Parcial2/13_lectura_de_archivo_de_parametros_con_errores.c

```
1  /*
2
3  Ultima modificacion :
4
5  Este programa lee un archivo de parametros que
6  contine comentarios y nombres de las variables
7
8  compile con : gcc 1_programa_basico_en_c_11.c -o 1_programa_basico_en_c_11.out
9
10 Ejecute con : ./1_programa_basico_en_c_11.out <archivo de parametros a leer> <
    simbolo de determina un comentario>
11
12 Entrada : nombre archivo de parametros (string), caracter que simboliza un
    comentario
13
14 Salida : Lectura del archivo de parametros en pantalla
15
16  */
17
18 // librerias de C
19 #include<stdio.h>
20 #include<stdlib.h> /*Error de sintaxis, la libreria deberia ser stdlib.h*/
21 #include<string.h>
22
23 // variables globales
24 int nLineasArchivo;
25 int tamanoString = 1000;
26
27 struct Parametros
28 {
29     char Infile[200];
30     char Outfile[200];
31     int Ncuerpos;
32     double tIntegracion;
33 }parametros /*Error de sintaxis; se debe usar ";" despues de definir la
    variable de estructura*/
34
35 // rutinas y funciones
36 int contador_de_lineas(char *infile);
37 int lee_linea_archivos(char *infile, char L[nLineasArchivo][tamanoString], char
    comentario, int ignorar[nLineasArchivo]);
38 int extrae_valores_parametros(char L[nLineasArchivo][tamanoString], char
    comentario, int ignorar[nLineasArchivo]);
39
40 // funcion principal
41 int main(int argc, char *argv) /*Error de sintaxis; [] tiene que ser la
    direccion de un punteros a caracteres*/
42 {
43     // recibe por linea de comandos el nombre del archivo de parametros
44     char *infile, *simboloComentario, comentario;
45     infile = argv[1];
46     simboloComentario = argv[2];
47
48     printf("Esta es la fila 0 de argv: %s\n",argv[0]);
49     printf("Leyendo archivo : %s\n",infile);
50     printf("Comentarios definidos por : %s\n",simboloComentario);
51     comentario = simboloComentario[0];
```

```
52     printf("Comentarios : %s\n",comentario); /*Error logico; es necesario que
sea %c*/
53
54     // calculo el numero de lineas que contiene el archivo
55     nLineasArchivo = contador_de_lineas(&infile); /*Error logico; no se
necesita el operador de dirección*/
56
57     printf("el archivo contiene %d lineas\n\n",nLineasArchivo);
58     char lineas[nLineasArchivo][tamanoString]; // matriz de lineas
59     int ignorar[nLineasArchivo]; // vector que controla si una linea es
ignorada
60
61     // lee todas las lineas del archivo de parametros
62     lee_linea_archivos(infile, lineas, comentario, ignorar);
63
64     // extrae los valores de los parametros desde las lineas leidas
65     extrae_valores_parametros( lineas, comentario, ignorar);
66
67     // Imprime los parametros
68     printf("\n\nParametros :\n");
69     printf("=====\n");
70     printf("parametros.Infile = %s\n",parametros.Infile);
71     printf("parametros.Outfile = %s\n",parametros.Outfile);
72     printf("parametros.Ncuerpos = %d\n",parametros.Ncuerpos);
73     printf("parametros.tIntegracion = %lf\n",parametros.tIntegracion);
74     printf("=====\n");
75     return 0;
76 } // fin funcion principal
77
78 // Esta funcion retorna de lineas que tiene un archivo
79 // contando la cantidad de saltos de linea almacenados en el archivo
80 // hasta que encuentra el final del archivo.
81 int contador_de_lineas(char *infile)
82 {
83
84     int nLineas = 0;
85     char c;
86     FILE *pf;
87
88     if( (pf=fopen(infile,"r")) == NULL )
89     {
90         printf("no puedo abrir archivo %s\n",infile);
91         exit(0);
92     }
93
94     while( (c=getc(pf)) != EOF ) /*Error sintaxis: se debe usar fgetc en vez de
getc*/
95     {
96         if( c=='\n' )
97             ++nLineas;
98     }
99     fclose(pf);
100     return nLineas;
101 }
102
103 // Esta funcion lee todas las lineas del archivo de parametros y la almacena
104 // en la matriz de linea, cada una es un string.
105 int lee_linea_archivos(char *infile, char L[nLineasArchivo][tamanoString], char
comentario, int ignorar[nLineasArchivo])
106 {
```

```
107     int i, j;
108     string c; /*Error de sintaxis; string es una variable de python*/
109     FILE *fInfile = fopen(infile,"rb"); /*Error error logico puede que no lea
en algunos compiladores*/
110
111     printf("Archivo leído:\n");
112     printf("=====\n");
113     for( i=0, i<nLineasArchivo, i++ ) /*Error de sintaxis; el inicio-parada-
paso deben estar sepadas por ;*/
114     {
115
116         j = ignorar[i] = 0;
117
118         // detecta lineas en blanco y debe ignorarse
119         if( (c = fgetc(fInfile)) == '\n' )
120         {
121             ignorar[i] = 1;
122             L[i,j] = c; /*Error de sintaxis; note que esta igualando un
array de caracteres a un caracter*/
123             printf(" - ignorar %d\n",ignorar[i]);
124             continue;
125         }
126         else
127         {
128             L[i][j++] = c;
129             // se debe ignorar las lineas que inicien con el simbolo de
comentario y el EOF
130             if( (c == comentario) or (c == EOF) ) /*Error sintaxis; se debe
usar ||*/
131                 ignorar[i] = 1;
132
133             while( (c = fgetc(fInfile)) != "\n" ) /*Error sintaxis,
comparacion con caracter*/
134                 L[i][j++] = c;
135         }
136
137         L[i][j] = '\0';
138         printf("%d - ignorar %s\n",L[i],ignorar[i]); /*Error sintaxis, imprimir
1 variable tipo char y otra tipo entero*/
139     }
140     printf("=====\n");
141
142     fclose(fInfile);
143     return 0;
144 }
145
146 // Esta funcion extrae los valores de los parametros desde las lineas leidas
147 int extrae_valores_parametros( char L[nLineasArchivo][tamanoString], char
comentario, int ignorar[nLineasArchivo])
148 {
149
150     int i, j, totalParametros = 4;
151     const int nParametro; /*Error logico, al ser constante no puede variar*/
152     char string1[300], string2[300], string3[300];
153     char nombreParametros[totalParametros][200];
154     void *apuntadorParametro[totalParametros];
155     int tipoParametro[totalParametros];
156
157     // Macros, esto lo veremos despues
158     #define INT 1; /*Error sintaxis, no son necesarios los puntos y coma*/
159     #define DOUBLE 2;
```

```
160     #define STRING 3;
161
162     // asocia los nombres de los parametros con las variables que los va a
    almacenar
163     nParametro == 0; /*Error logico, == es para comparar, no igualar*/
164
165     strcpy(nombreParametros[nParametro], "Infile");
166     apuntadorParametro[nParametro] = &parametros.Infile;
167     tipoParametro[nParametro++] = STRING;
168
169     strcpy(nombreParametros[nParametro], "Outfile");
170     apuntadorParametro[nParametro] = &parametros.Outfile;
171     tipoParametro[nParametro++] = STRING;
172
173     strcpy(nombreParametros[nParametro], "Ncuerpos");
174     apuntadorParametro[nParametro] = &parametros.Ncuerpos;
175     tipoParametro[nParametro++] = INT;
176
177     strcpy(nombreParametros[nParametro], "tIntegracion");
178     apuntadorParametro[nParametro] = &parametros.tIntegracion;
179     tipoParametro[nParametro++] = DOUBLE;
180
181     printf("\n\nLineas leidas a tramos\n");
182     printf("=====\n");
183     for( i=0; i<nLineasArchivo; i++ )
184     {
185         if( ignorar[i] == 0 )
186         {
187             sscanf(L[i], "%s%s%s", string1, string2, string3); /*Error; error de
    sintaxis, sobra un ;*/
188             printf("string1 = %s - string2 = %s - string3 = %s\n", string1, string2,
    string3);
189
190             // encuentra el parametro en la lista de parametros
191             for( j=0; j<totalParametros; j++ )
192                 if( strcmp(string1, nombreParametros[j]) == 0 ) /*Error de sintaxis,
    el operador es ==*/
193                     break;
194
195             // asigna el parametro a la variable correspondiente
196             switch ( tipoParametro[j] )
197             {
198                 case INT:
199                     *((int *) apuntadorParametro[j]) = atof(string2); /*Error
    logico, se iguala un entero a un double*/
200                     break;
201                 case DOUBLE:
202                     *((double *) apuntadorParametro[j]) = atof(string2);
203                     break;
204                 case STRING:
205                     strcpy(apuntadorParametro[j], string2);
206                     /*Error logico: falta un break*/
207             }
208
209         }
210     }
211     printf("=====\n");
212     return 0;
213 }
214
```

```
215 /*A continuacion se enlistan los errores que presenta el código*/
216
217 /*
218 - Línea 20: error de sintaxis, falta una "d" porque en principio la librería
que se debería llamar es "stdlib.h".
219 - Línea 33: error de sintaxis ";", justo después de definir la variable de
estructura "parametros;".
220 - Línea 41: error de sintaxis "[ ]" en llevar la dirección de memoria del
puntero a un carácter entonces no es posible que sea solo char *argv, necesita
ser "char *argv[ ]".
221 - Línea 52: error lógico. Cuando se definen las variables tipo char es
necesario que tenga comentario no es un puntero a carácter por tanto no se
puede "%s", se debe usar "%c".
222 - Línea 55: error lógico. Note que la función contador de líneas va a trabajar
con un puntero de un puntero (lee el archivo como una matriz, avanzando por las
filas) a carácter, el operador de dirección no se puede porque va a dirección y
no al valor que hay almacenado en esa dirección. Sobra el operador "&".
223 - Línea 94: se busca leer un carácter como un entero, por ello es necesario el
uso de "fgetc()", "getc()".
224 - Línea 108: error de sintaxis. Note que "string" es un tipo de variable en
python, en este caso se debe definir como "char".
225 - Línea 109: error lógico. Si bien algunos compiladores logran leer en binario
el archivo, no todos pueden hacerlo. Por lo tanto, es necesario leer y no leer
precisamente en binario "r", en vez de "rb".
226 - Línea 113: error de sintaxis. El principio, la condición de parada y el paso,
deben estar separadas por ";", no por ":".
227 - Línea 122: error de sintaxis. No es posible igualar
228 "L[i,j]" a c porque es igualar un vector de caracteres a un solo carácter.
Esta notación es válida en Python, pero en particular en C, es necesario
considerar "L[i][j]".
229 -Línea 133: error de sintaxis. No se puede usar dobles comillas para comparar
"(c=fgetc(fInfile)) != '\n'", porque se está comparando solo con un carácter no
con una cadena.
230 - Línea 138: error de sintaxis. Note que "ignorar" es una variable entera y "L"
es tipo char. Entonces es necesario cambiar %s y %d.
231 - Línea 151: error lógico. Si se define como una constante entera "nParametro",
no se puede variar o aumentar su valor más adelante, por tanto basta con
definirla como "int".
232 - Línea 158,159,160: error de sintaxis que cuenta por 3. No se deben colocar
los puntos y comas al final de la definición de los macros.
233 - Línea 163: error lógico. Se inicia el nParametros en 0, para esto basta con
"=". Dos implican una comparación.
234 - Línea 187: error de sintaxis. Sobra una elemento "%s", pues solo son tres
variables tipo char *.
235 - Línea 192: error de sintaxis. El operador de comparación de un elemento con
otro en un condicional está dado por "==", no solo por "=".
236 - Línea 199: error lógico. El puntero hace un casting a entero entonces se debe
igualar a un entero, pero se está pasando string2 a un double, en vez de a un
entero. Por eso se usa "atoi()" en lugar de "atof()".
237 - Línea 206: falta un break en el switch.
238 */
```